

Introducción a JSON

Sobre JSON

Nota: el contenido de esta práctica se basa en el tutorial publicado en [Trabajando con datos JSON en Python](#).

Desde su inicio, JSON se ha convertido rápidamente en el estándar de facto para el almacenamiento e intercambio de información. Por un lado, es un formato de datos utilizado para representar y almacenar datos estructurados. Además, es muy común transmitir y recibir datos entre un servidor y una aplicación web en formato JSON. No es más que un formato estandarizado que la comunidad usa para transmitir datos. Tenga en cuenta que JSON no es el único formato disponible para este tipo de trabajo, pero XML y YAML son probablemente los únicos que vale la pena mencionar.

JSON (JavaScript Object Notation) se inspiró en un subconjunto del lenguaje de programación JavaScript. Sin embargo, no se preocupe: JSON se ha convertido desde hace mucho tiempo en un lenguaje independiente y existe como su propio estándar, por lo que afortunadamente podemos evitar tener que aprender JavaScript.

Finalmente, la comunidad en general adoptó JSON porque es fácil para los humanos y para las máquinas.

Un ejemplo de JSON

A continuación se muestra un posible ejemplo de JSON:

```
{
  "firstName": "Jane",
  "lastName": "Doe",
  "hobbies": ["running", "sky diving", "singing"],
  "age": 35,
  "children": [
    {
      "firstName": "Alice",
      "age": 6
    },
    {
      "firstName": "Bob",
      "age": 8
    }
  ]
}
```

Como puede ver, JSON admite tipos primitivos, como cadenas y números, así como listas y objetos anidados.

¡Espera, eso parece un diccionario de Python! Es una notación de objeto prácticamente universal en este momento y se utiliza en muchos lenguajes actuales. Os animo a buscar las diferencias.

Usando JSON desde Python

Python viene con un paquete incorporado llamado json para codificar y decodificar datos JSON. Para trabajar con JSON (cadena o archivo que contiene un objeto JSON), puede usar el dicho módulo de Python. Debe importar el módulo antes de poder usarlo. Para ello tenemos que incorporar esta línea en la parte superior de su archivo Python:

```
import json
```

Ahora veamos el código Python que lee JSON de una variable y lo transforma en Python.

Ejemplo 1: Python que lee JSON

```
import json

person = '{"name": "Bob", "languages": ["English", "Fench"]}'
person_dict = json.loads(person)

# Output: {'name': 'Bob', 'languages': ['English', 'Fench']}
print( person_dict)

# Output: ['English', 'French']
print(person_dict['languages'])
```

Puede analizar una cadena JSON utilizando el método json.loads() El método devuelve un diccionario. Observar que la variable "person" almacena una variable de tipo string. Ella no tiene "información" o "datos" en el sentido Python. En cambio, la variable "person_dict" ya es un diccionario Python con todas las de la Ley. Ya se puede consultar información sin problemas.

Ejemplo 2: Python que lee JSON desde un fichero

Ahora vamos a transformar el ejemplo anterior para que los datos JSON procedan de un fichero. Tenerlos almacenados en una variable no parece demasiado útil.

Primero cree un fichero llamado "person.json" e introduzca la siguiente información:

```
{ "name": "Bob",
  "languages": ["English", "Fench"]
}
```

Es importante tener cuidado con las comillas.

Ahora viene el código Python que lee ese fichero y lo procesa:

```
import json

with open('path_to_file/person.json') as f:
    data = json.load(f)

# Output: {'name': 'Bob', 'languages': ['English', 'Fench']}
print(data)
```

Hemos utilizado el método `json.load()` para leer un archivo que contiene un objeto JSON. Observar que es diferente `json.load()` de `json.loads()`. ¿Qué diferencia hay?

Ejemplo 3: de Python a JSON

Lo contrario del ejemplo 1 sería convertir un diccionario de Python en una cadena JSON. Es muy fácil. Para ello puede utilizar el método `json.dumps()`.

```
import json

person_dict = {'name': 'Bob', 'age': 12, 'children': None}
person_json = json.dumps(person_dict)

# Output: {"name": "Bob", "age": 12, "children": null}
print(person_json)
```

De nuevo, la variable `person_dict` es un diccionario Python con la que podemos manipular información mientras que `person_json` es una variable con un string.

Ejemplo 4: de Python a un fichero con JSON

En este ejemplo hacemos lo contrario del ejemplo 2. Tenemos un diccionario en JSON que vamos a almacenar en un fichero.

```
import json

person_dict = {"name": "Bob",
"languages": ["English", "Fench"],
"married": True,
"age": 32
}
```

```
with open('person.txt', 'w') as json_file:  
    json.dump(person_dict, json_file)
```

En el programa anterior, hemos abierto un archivo llamado person.txt en modo de escritura usando 'w'. Si el archivo aún no existe, se creará. Luego, con el método json.dump() se transforma person_dict en una cadena JSON que se guardará en el archivo.

Cuando ejecute el programa, se creará el archivo person.txt. Ejecuta el código y observa el contenido del fichero. ¿Qué tiene?

Tenga en cuenta que json.dump() toma dos argumentos posicionales: (1) el objeto de datos que se va a serializar y (2) el objeto tipo archivo en el que se escribirán los bytes.

De nuevo, ¿puede decir la diferencia entre json.dump u json.dumps?

Sobre Serialización

En los ejemplos anteriores hemos visto un proceso llamado "Serialización". El proceso que transforma datos (variables) en formatos externos (normalmente en strings) se llama serialización. Hay varios formatos y JSON es uno de ellos. Otros formatos son XML o YAML.

El término de serialización se refiere por tanto a la transformación de datos en una serie de bytes (que pueden leerse o escribirse en serie) para ser almacenados o transmitidos a través de una red. Naturalmente, la deserialización es el proceso recíproco de decodificación de datos que se ha almacenado o entregado en el estándar JSON.

En resumen: a codificación (serialización) es para escribir datos en el disco o de la red, mientras que la decodificación (deserialización) es para leer datos en la memoria.

Ejercicios

Ejercicio 1:

Diseña un diccionario que tenga el nombre y cuenta de gitlab de dos o tres compañeros de clase. Haz un programa Python que guarde esos datos en un fichero JSON.

Ejercicio 2:

Intercambia el fichero generado en el paso anterior con algún compañero de clase. Haz un programa Python que lea ese fichero e imprima por pantalla y en este orden: primero todos los nombres que hay y a continuación, todas las cuentas gitlab. Para terminar, haz que tu programa imprima por pantalla el número de personas que tiene el fichero.